

Cotangent: Preserving Reduction Arithmetic in Batched Gradient Handling

Atila Vahedian

October 2026

Abstract

Gradient handling can occupy a substantial fraction of small-model training on an accelerator even when the optimizer is already fused. Batching this work is straightforward in real arithmetic but can change floating-point reductions. We study a gradient layout that preserves the native two-level clipping norm while batching equal-length tensors on PyTorch’s Metal backend. The method retains every model derivative and applies native fused AdamW to consistently packed parameter and gradient coordinates. An independently frozen experiment with 100 paired seeds reduces time to a fixed validation target by 15.03% (99.5% confidence interval, 13.50–16.56%) relative to eager native fused AdamW, while meeting a predeclared 0.01-bit-per-byte quality tolerance. A separately frozen 128-run extension isolates norm batching from packing and evaluates four configurations against training-only-selected compiled baselines. Using the same compiled model, mean training-time reductions range from 23.55% to 34.07%. The extension’s descriptive quality intervals do not establish the 0.01-BPB tolerance for every configuration. Numerical checks cover complete model updates, optimizer states, reduction boundaries and composition with the compiler. The results are specific to one Apple M5 Pro, the pinned PyTorch backend and byte language models; the study does not establish performance on other accelerators or model scales.

1 Introduction

The work after backward is easy to underestimate. A training step must combine gradients to determine a clipping coefficient, scale them, update moment estimates, and change parameters. A fused optimizer reduces part of this cost, but the surrounding norm calculations and tensor dispatch can remain visible when a model consists of many modestly sized parameter tensors.

Cotangent investigates that remaining cost. The starting point is a conventional byte transformer with native attention, linear and embedding derivatives. Its clipping operation evaluates 37 individual parameter norms before combining them. The proposed layout batches those norms into four equal-length groups and exposes one contiguous coordinate vector to native fused AdamW.

The difficulty is numerical. A flat global norm has the same mathematical value as a norm of parameter norms, yet the two computations can use different floating-point additions. Even preserving a separate row norm is insufficient if tensor shape selects a different kernel. We therefore preserve the native two-level norm structure and choose a layout that retains the pinned MPS generic reduction path. Correctness is checked before timing is interpreted.

Two experiments serve different purposes. The first is a powered replication with a fixed time-to-quality target and quality tolerance. The second extends the evidence with a factorial ablation, additional model layouts, and stronger compiler baselines. In the extension, both arms use the same selected model execution path. This isolates the additional effect of gradient handling from

the benefit of compilation itself. Code, protocols, observations and analysis are available at <https://github.com/atilavahedian/cotangent>.

2 Method

2.1 Preserve the clipping computation

Let $g_\ell \in \mathbb{R}^{K_\ell}$ be the completed gradient of parameter tensor ℓ . For a clipping threshold τ , native clipping computes

$$n_\ell = \|g_\ell\|_2, \quad n = \left(\sum_{\ell=1}^L n_\ell^2 \right)^{1/2}, \quad c = \min \left(1, \frac{\tau}{n + \epsilon_c} \right), \quad h_\ell = c g_\ell, \quad (1)$$

where $\tau = 1$ and $\epsilon_c = 10^{-6}$ in these experiments. We retain a separate norm for every parameter and retain the original order in the final reduction. No gradient coordinate is sampled, dropped or approximated.

For a bucket containing B gradients of length K , stack their flattened coordinates and view the resulting tensor as $B \times (K/2) \times 2$. Reducing the two trailing axes produces B independent parameter norms. For even $K \geq 4$, both reduced axes are nontrivial. Short or odd lengths use separate native norms. Scalar outputs are restored to parameter order before the final norm in Equation 1.

This choice follows a specific dispatcher condition [7]. At the pinned PyTorch revision, the MPS inner norm fast path requires multiple output elements and exactly one nontrivial reduced dimension on the innermost axis. A conventional $B \times K$ row reduction meets those conditions. The two-axis layout avoids that path and invokes the generic norm kernel used by a native scalar parameter norm. Each parameter keeps K reduced elements and the same threads per group. The source-level argument concerns this particular implementation; it is supported by measured numerical checks, rather than asserted for arbitrary backends or versions.

2.2 Pack coordinates consistently

The model retains independent autograd leaves. Before its first forward pass, their storage is attached to disjoint views of a master parameter vector. Backward still materializes every native parameter gradient. The completed gradients are concatenated, multiplied by c , and passed to native fused AdamW. The tensor copies, concatenation and temporary allocations are part of the measured training time.

The supported setting has one hyperparameter group, homogeneous contiguous parameters, dense gradients for all active coordinates, and independent, nonoverlapping parameter storage. Parameters must not be moved or reassigned after construction. Partial updates, mixed groups and sparse gradients require additional treatment and are outside the implemented experiment.

Proposition 1 (Coordinate equivalence in real arithmetic). *Let P be the fixed reindexing that concatenates all active coordinates. With identical initialization, hyperparameters, scalar step counts and complete gradients, clipping followed by AdamW commutes with P in real arithmetic.*

Proof. The reindexing is an isometry, so $\|Pg\|_2 = \|g\|_2$ and the scalar clipping coefficient is unchanged. Let $h = cg$. AdamW’s moments satisfy

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) h_t, \quad (2)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) h_t^2. \quad (3)$$

Reindexing commutes with coordinatewise addition, multiplication, squaring, square roots and division. Bias correction uses the same scalar t . Consequently the decoupled weight decay and adaptive update also commute with P . Induction from the same initial parameters and zero moments gives coordinate-equivalent iterates. $\square$

This algebra does not establish bitwise equality of parallel reductions. Floating-point addition is not associative. The implementation therefore keeps the native reduction structure and tests the remaining finite-precision behavior on the actual device. Those checks also distinguish optimizer equivalence from run-to-run variation in independently executed native embedding accumulation.

3 Experimental design

3.1 Environment, models and data

All GPU measurements use one Apple M5 Pro with 24 GiB RAM, macOS 27.0, Python 3.12.14 and PyTorch 2.14.1 MPS. The Torch source revision is `5c4886908584029761b579af026dcfb627c84070`. Weights and completed gradients are FP32; forward computation uses BF16 autocast. AdamW uses a peak learning rate of 10^{-3} , $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, weight decay 0.1 and global clipping at 1. The learning rate warms up and then follows a cosine schedule to one tenth of the peak. Batch size is eight and sequence length is 256.

The corpus is a pinned byte serialization of WikiText-2 [8]. The first 95% of training bytes is used for optimization; the remaining 5% is a diagnostic split. Full official validation and test evaluations cover 1,148,006 and 1,292,012 predicted bytes respectively. These corpus splits were exposed during development and are explicitly reused as quality regression data. Fresh seed pairs do not make the corpus an unseen generalization test. All models train from scratch and use an untied output head.

Model	Parameters	Width	Layers	Tensors / buckets
Small transformer	3,346,944	256	4	37 / 4
Medium transformer	19,280,896	512	6	53 / 5
Gated GQA	9,737,088	384	6	52 / 4
Causal convolution	3,746,816	256	6	53 / 4

Table 1: Model configurations in the extension. Bucket counts refer to equal-length completed gradient tensors, including norm parameters.

The two original transformers use pre-norm attention and GELU feed-forward layers. The additional decoder uses six query heads, two key/value heads, RMSNorm and a SwiGLU-style gated feed-forward layer [9, 11, 10]. Its key/value heads are explicitly repeated for the native attention call. The convolutional model uses causal, depthwise, kernel-three convolutions with dilations 1, 2, 4, 8, 16, 32, residual pointwise mixing and GELU feed-forward layers, following the causal dilated design of WaveNet [12]. Architecture-level language modeling quality is not the comparison objective: each optimizer treatment is compared within the same architecture.

3.2 Independent time-to-quality replication

The confirmatory experiment (V5) contains 100 paired seeds and 200 runs of the 3.35M transformer, each trained for 2,000 steps. Every pair matches initialization, data schedule, architecture, precision

and optimizer settings. Exactly 50 blocks run native first and 50 run Cotangent first; block order is shuffled before training. The protocol is public at tag `protocol-v5`.

The quality target is 3.2 bits per byte (BPB) on a scheduled validation probe covering 65,536 targets. Probes occur every 100 steps. The target time is the first observed crossing, with no interpolation. Its clock includes setup, priming, transfer, forward/backward, gradient handling, optimizer updates, equal host/resource checks and preceding probes. Python process launch is excluded equally. Every training step synchronizes MPS.

The fixed decision requires at least 10% mean paired target-time reduction, a positive timing lower bound, and an upper full-test degradation bound no greater than 0.01 BPB. Intervals are two-sided 99.5% paired Student- t intervals. The sample size was fixed before V5 outcomes using all 25 quality differences from the preceding V4 study, whose standard deviation was 0.0209261 BPB. A normal planning approximation at zero degradation and 90% power gives 73.2 pairs; the protocol rounds up to 100. V4’s failed combined decision remains unchanged. V5 changes neither the method nor its thresholds.

3.3 Ablations and stronger execution paths

The extension (V6) is a separately frozen descriptive study with 128 runs. Eight seed blocks cross scalar/bucketed norms with native/packed fused AdamW, using 800 steps per cell on the small transformer. A rotating Latin order puts each of the four cells in each execution position twice. This separates norm batching, parameter packing and their interaction.

The breadth study uses twelve paired seeds per configuration, 1,200 steps per run, and balanced six-AB/six-BA execution orders. It screens native fused, automatic and loop AdamW, direct native multi-tensor norm/multiply operations, TorchInductor, and AOT eager capture using two training-only diagnostic seeds. Selection uses mean training time after the first 20 pilot steps. TorchInductor is the fastest eligible baseline in all four configurations. Compilation uses `fullgraph=True` and static shapes. Both official arms therefore use the same compiled forward/backward; Cotangent changes the subsequent norm/layout/update handling.

Compilation and setup are reported separately and are included in total completion time. Process state is fresh for every run; generated compiler artifacts use a shared cache already exercised by pilots. These are warm-cache measurements, not a characterization of first-ever compilation latency. Gradient handling and native fused AdamW remain outside the compiled model call. The study does not exhaust every compiler/optimizer configuration.

All V6 blocks are shuffled before outcomes and the sample sizes are fixed. V6 reports per-configuration descriptive 99% paired Student- t intervals; it does not introduce a new powered quality gate or simultaneous coverage across every comparison. Official validation/test data are evaluated once after each run and are not used to select the execution paths.

For paired times T_i^{base} and T_i^{cot} , the reported reduction is the mean of $s_i = 1 - T_i^{\text{cot}}/T_i^{\text{base}}$. It is not the ratio of aggregate mean times. For paired differences z_i , the interval is $\bar{z} \pm t_{n-1, 1-\alpha/2} s_z / \sqrt{n}$. Full-test quality differences use candidate minus baseline BPB; positive values indicate worse quality. All observations, including adverse pairs, are retained.

Sampled process RSS is capped at 5 GiB, MPS driver allocation at 8 GiB, and swap growth at 512 MiB. Checks occur every ten steps. Training runs are serial; no task-owned GPU jobs execute concurrently.

4 Results

4.1 Numerical behavior

The original check covers 32 full-model updates: two model sizes, FP32/BF16 forward precision and eight consecutive steps. Losses, gradients, clipping norms, parameters and both moment buffers match bitwise. Seven additional full-training derivative-controlled pairs finish with identical whole-model hashes and full evaluation scores. Both arms use matched deterministic embedding derivatives in these controls to isolate optimizer arithmetic. Primary performance comparisons retain native embedding backward.

The extension adds 128 full-model checks across all four configurations, two forward precisions, four gradient-handling treatments and four consecutive updates. All checked losses, gradients, norms, parameters and moments match the native clipping/fused AdamW reference. A further 16 BF16 checks compare the two compiled paths with matched deterministic derivatives; losses, gradients, norms and resulting parameter hashes match. These compiled checks compare identical compiler paths, not compiled arithmetic with eager arithmetic.

A norm sweep checks 32 lengths, bucket sizes 1, 3 and 8, and FP32, FP16 and BF16 gradient tensors: all 288 total norms match bitwise on this backend. It includes short/odd fallback lengths and values near reduction-size boundaries. Missing gradients, mixed parameter types and overlapping parameter objects are rejected. These observations do not establish equivalence for every tensor stride, nonfinite input, compiler transformation or future backend version.

4.2 Confirmatory replication

All 200 V5 runs complete and both arms reach the target in every pair. Cotangent reduces target time by 15.03% (99.5% CI, 13.50–16.56%) and fixed-budget training time by 16.97% (16.26–17.69%). Average target times are 16.65 seconds for native and 14.13 seconds for Cotangent; average training times are 34.84 and 28.92 seconds. Ninety-nine pairs reach the target faster with Cotangent; the one slower pair remains in the analysis.

Mean full-test degradation is 0.00138 BPB (99.5% CI, -0.00451 to 0.00728). Its upper bound is below the predeclared 0.01 tolerance. This supports the bounded noninferiority decision, rather than improved model quality. Fifty of the 100 pairs have worse candidate test BPB.

4.3 Factorial ablation

Norm batching alone reduces training time by 15.92%, packing alone by 2.16%, and their combination by 17.86%. The main-effect and interaction contrasts in Table 2 use the complete four-cell block. They describe this architecture and execution regime; an additive decomposition is not assumed.

Contrast	Training-time reduction (%)	99% CI
Norm batching only	15.92	15.50–16.33
Packing only	2.16	1.27–3.05
Combined	17.86	16.92–18.80
Norm main effect	15.81	15.25–16.36
Packing main effect	2.05	1.35–2.75
Signed interaction	0.22	-0.65–1.08

Table 2: Eight paired factorial blocks, 800 steps per cell. Single-factor contrasts compare with scalar norms/native layout. Main effects average the factor’s two conditional effects and normalize by the block’s reference time. The signed interaction is $(T_{00} - T_{10} - T_{01} + T_{11})/T_{00}$.

4.4 Composition with compiled training

With the same compiled model in each pair, mean training-time reductions range from 23.55% to 34.07%. Every per-configuration descriptive timing interval has a positive lower bound. Table 3 reports both absolute times and paired reductions. These results extend the implementation evidence across layouts; they do not replace the independent time-to-quality decision in V5.

Model	Base / Cot. time (s)	Reduction (%)	99% CI
Small transformer	15.51 / 11.86	23.55	22.97–24.13
Medium transformer	56.32 / 37.16	34.07	32.87–35.26
Gated GQA	35.03 / 23.99	31.55	30.73–32.37
Causal convolution	17.24 / 12.54	27.30	26.95–27.64

Table 3: Fixed-budget training, twelve pairs per model, 1,200 steps per arm. Every baseline uses compiled forward/backward plus native clipping and fused AdamW. Every candidate uses the same compiled model plus Cotangent handling. Intervals are descriptive per configuration.

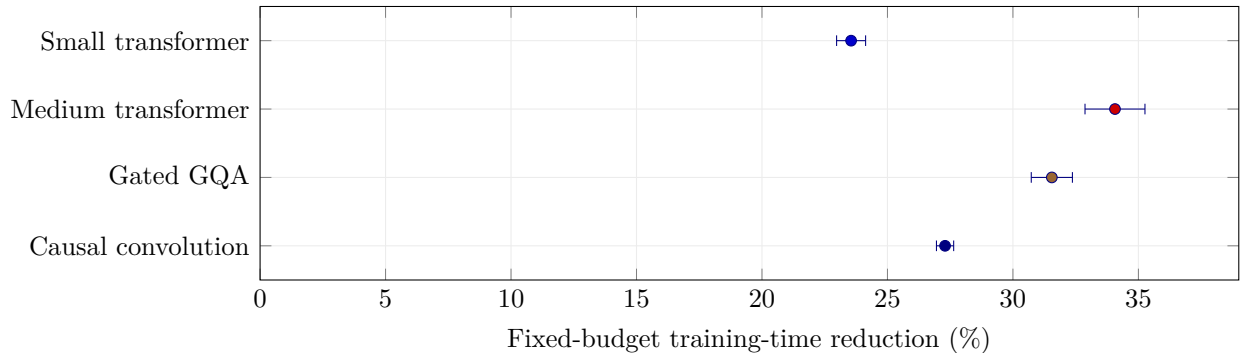


Figure 1: Additional benefit of gradient handling on top of the same compiled model. Points are means of paired reductions; horizontal bars are descriptive 99% confidence intervals.

Model	Base / Cot. test BPB	Difference	99% CI
Small transformer	3.0583 / 3.0620	+0.00365	-0.00940 to +0.01670
Medium transformer	2.8992 / 2.8854	-0.01382	-0.07474 to +0.04711
Gated GQA	3.0413 / 3.0685	+0.02723	-0.00819 to +0.06264
Causal convolution	2.3100 / 2.3100	-0.00000	-0.00005 to +0.00005

Table 4: Full-test quality in the extension. Difference is candidate minus baseline BPB; positive is worse. These descriptive intervals do not replace the independent V5 quality decision.

The descriptive quality interval extends above the original 0.01-BPB tolerance for Small transformer, Medium transformer, Gated GQA. The extension therefore does not establish that narrow quality tolerance for every configuration. The controlled update checks and the V5 quality result remain separate evidence. Across the 48 breadth pairs, 0 have slower candidate training and 31 have worse candidate full-test BPB. Both counts remain in the analysis. Gated GQA has a mean test difference of +0.02723 BPB, with worse candidate test scores in 10 of twelve pairs. The controlled arithmetic checks do not explain this observed difference. The causal convolution’s upper bound is 0.00004695 BPB. Broader quality equivalence therefore requires further investigation, rather than an assumption that every difference is harmless accumulation noise.

Model	Total-time reduction (%)	99% CI	Driver MiB: base / Cot.
Small transformer	17.89	16.70–19.09	219 / 1075
Medium transformer	28.73	27.61–29.86	1163 / 1139
Gated GQA	25.28	24.36–26.20	1163 / 1099
Causal convolution	19.61	18.88–20.35	180 / 1068

Table 5: Total completion time includes setup, diagnostics, full evaluation and checkpoint writing. Driver values are the largest sampled allocations per arm, including allocator/cache effects; they are not a continuous live-tensor trace.

The largest sampled driver allocation across breadth baselines is 1.14 GiB, versus 1.11 GiB for candidates. Table 5 shows the substantial increase in sampled driver allocation for the small transformer and causal convolution candidates. These values include allocator/cache effects and are not a continuous live-tensor memory trace. The additional buffers are part of the implementation and may matter more at larger scales. Total completion times, including setup, diagnostics, full evaluation and checkpoint writing, are also reported in the public paired records. Every one of the 128 extension snapshots is restored locally. Weight-file hashes, reconstructed parameter hashes and finite CPU inference are verified. This audit does not repeat the official test evaluation. All weights remain local; public records contain the hashes needed to identify the measured runs.

5 Related work

AdamW’s coordinatewise moment and decoupled-decay updates provide the optimizer semantics used here [1]. Parameter flattening is also a standard training-systems mechanism, including PyTorch FSDP [4]. DeepSpeed FusedAdam and NVIDIA Apex’s multi-tensor norm implementation batch work across parameter tensors [5, 6]. The present study evaluates a specific Metal execution layout and its numerical behavior; CUDA source implementations are related mechanisms, not measured baselines on the available hardware.

Reproducible summation has a substantial numerical-analysis literature. Ahrens et al. construct order-independent accumulators [2]. The implementation here instead retains the reference reduction behavior for tested shapes on a pinned backend. TorchDynamo and TorchInductor reduce Python and kernel overhead through graph compilation [3]; the extension shows whether gradient handling provides an additional benefit with that execution path. The experiment compares these components directly rather than attributing the compiler’s gains to the gradient layout.

6 Discussion and limitations

The factorial evidence attributes the larger single-factor saving to norm batching. This locates the benefit within the measured training step rather than inferring it from a synthetic optimizer benchmark. Compiler screening also changes the relevant baseline: on the small-model pilot, compiled native training is faster than the original eager candidate. The extension therefore composes the gradient layout with that same compiler path. Its comparison measures the added effect of handling completed gradients, while leaving compiler arithmetic common to both arms. The saving is specific to the measured workload. Larger derivative matrix products can reduce the fraction of time spent handling gradients, and more distinct parameter lengths can reduce the opportunity for norm batching. Conversely, dispatch overhead can be prominent when many small parameter tensors are present. The study does not establish a hardware-independent relationship between parameter count and speedup.

Numerical equality is also scoped. The real-arithmetic coordinate argument is general under its assumptions, but the reduction matching depends on an actual kernel implementation. Dense contiguous gradients, one hyperparameter group and complete updates are enforced in this experiment. Shared or overlapping parameter objects, sparse gradients, distributed synchronization, alternative precision regimes and different backend versions need independent treatment. Native embedding accumulation remains execution-sensitive; controlled equality does not imply independently executed primary runs have identical final weights.

The experiment uses one Apple device and one reused corpus. V6 has twelve pairs per configuration and descriptive quality intervals. Compiler artifacts are warm-cached; cold compilation costs and other compiler/optimizer settings remain uncharacterized. Both studies use per-step synchronization, which differs from an asynchronously queued production training loop. The reported effect should therefore be remeasured in the target deployment before extrapolation. No conclusion is drawn about CUDA, distributed training, billion-parameter models or model capability gains.

7 Conclusion

Cotangent batches gradient handling while retaining the native clipping structure and complete model derivatives. On the measured eager MPS setup, the independent replication meets its fixed time and quality requirements. The extension identifies the contribution of individual components and tests the method against executable compiled baselines on additional model layouts. Its practical lesson is to check the actual reduction arithmetic when changing tensor dispatch: mathematical equality alone does not certify numerical equivalence, and optimizer fusion alone does not remove all post-backward cost.

Reproducibility

The public repository contains the frozen protocols, complete raw curves, per-step timings, source fingerprints, model/schedule hashes, analysis and this standalone LaTeX source. Tags `protocol-v5` and `protocol-v6` precede their official outcomes. Public records-only analysis recomputes statistics without downloading data or weights; it explicitly leaves checkpoint-file verification unperformed. All five earlier studies remain in the experimental record, including their failed decisions. No observations are removed to improve a reported result. The paper and portable report are included in release `v0.6.0-research`.

References

- [1] I. Loshchilov and F. Hutter. Decoupled Weight Decay Regularization. ICLR, 2019. <https://arxiv.org/abs/1711.05101>.
- [2] W. Ahrens, J. Demmel, and H. D. Nguyen. Algorithms for Efficient Reproducible Floating Point Summation. ACM Transactions on Mathematical Software, 46(3), Article 22, 2020. <https://doi.org/10.1145/3389360>.
- [3] J. Ansel et al. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. ASPLOS, 2024. <https://docs.pytorch.org/assets/pytorch2-2.pdf>.
- [4] PyTorch contributors. FSDP flat parameter implementation. https://github.com/pytorch/pytorch/blob/main/torch/distributed/fsdp/_flat_param.py.
- [5] DeepSpeed contributors. FusedAdam implementation. https://deepspeed.readthedocs.io/en/latest/_modules/deepspeed/ops/adam/fused_adam.html.
- [6] NVIDIA Apex contributors. Multi-tensor L2 norm kernel. https://github.com/NVIDIA/apex/blob/master/csrc/multi_tensor_l2norm_kernel.cu.
- [7] PyTorch contributors. MPS norm dispatcher, revision 5c4886908584029761b579af026dcfb627c84070. <https://github.com/pytorch/pytorch/blob/5c4886908584029761b579af026dcfb627c84070/aten/src/ATen/native/mps/operations/ReduceOps.mm>.
- [8] S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer Sentinel Mixture Models. ICLR, 2017. <https://arxiv.org/abs/1609.07843>.
- [9] J. Ainslie et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. EMNLP, 2023. <https://arxiv.org/abs/2305.13245>.
- [10] N. Shazeer. GLU Variants Improve Transformer. 2020. <https://arxiv.org/abs/2002.05202>.
- [11] B. Zhang and R. Sennrich. Root Mean Square Layer Normalization. NeurIPS, 2019. <https://arxiv.org/abs/1910.07467>.
- [12] A. van den Oord et al. WaveNet: A Generative Model for Raw Audio. 2016. <https://arxiv.org/abs/1609.03499>.

A Implementation outline

1. Construct the model and attach its independent parameter leaves to disjoint master-vector views before any forward pass.
2. Run the selected native or compiled forward/backward and materialize every completed parameter gradient.
3. Group gradients by element count. Compute matched two-axis bucket norms, using native scalar fallbacks for short or odd lengths.
4. Restore scalar norms to parameter order, compute the original joint norm, and form the clipping coefficient.
5. Concatenate complete gradients, scale the vector, and apply native fused AdamW with the original hyperparameters and scalar step count.

The reference code is `research/v4/bucketed.py`. The extension’s factorial treatments and compiler composition are in `research/v6/methods.py`. All prior source freezes remain unchanged.

B Experimental record

Study	Method or purpose	Runs	Decision / scope
V1	Sampled weight gradients	36	Failed time and quality
V2	Packing with flat norm	24	Failed combined gate
V3	Deterministic adjoints, flat norm	41	Failed combined gate
V4	Matched bucket norms	70	Failed quality uncertainty
V5	Fixed-size independent replication	200	Passed fixed gates
V6	Factorial and compiled extension	128	Descriptive comparisons

Table 6: Official planned training observations. Diagnostic pilots and numerical verification steps are separate. V4 remains failed despite its timing advantage; V5 is an independent fixed-size study of the unchanged method.

The V4 upper 99% quality bound was 0.01160035 BPB, above the original 0.01 margin. Its failure motivated power planning, not a relaxed threshold. Stronger interval levels in later studies do not provide universal family-wise coverage for every exploratory choice made during development.